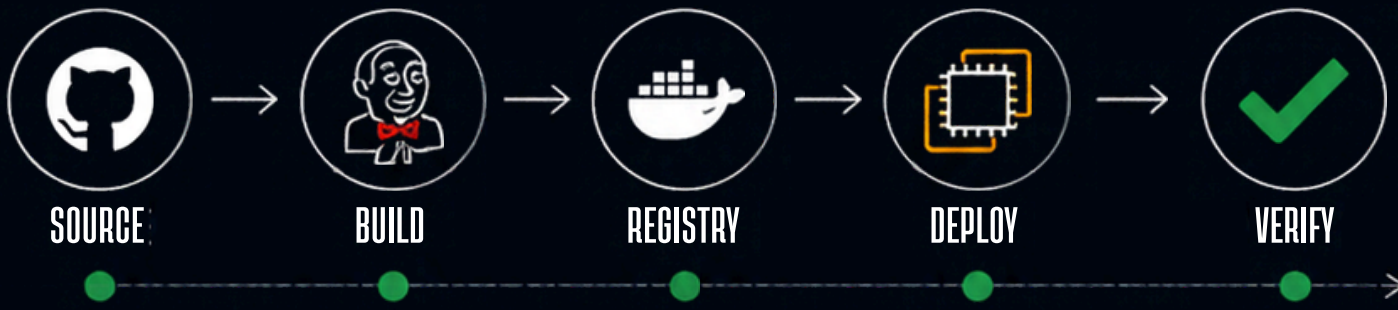
# NETPIX CI/CD PIPELINE
# AWS EC2 DEPLOYMENT

## Portfolio technical guide

Source-based revision: no commands or results are claimed as executed.

This guide documents the design and configuration of a CI/CD pipeline that automates building a containerized web application and deploying it to an AWS EC2 instance. It covers the end-to-end flow from source code to a verified deployment behind Nginx.




### SCOPE OF THIS GUIDE

01 Automated build and containerization

02 Automated deployment to AWS EC2

03 Verification and operational readiness

### TOOLCHAIN

GitHub · Jenkins · Docker · Docker Hub · Amazon EC2 · Nginx

### WHAT THIS GUIDE COVERS

Project overview · Architecture · Prerequisites · AWS setup · Jenkins configuration · Repository and credentials · Freestyle job · Build/deploy script · Verification · Troubleshooting

### SOURCE INTEGRITY

Facts are retained from the supplied PDF.
Conflicts and missing details are flagged for confirmation.

Author: Ritesh Thorat

# 02 //END-TO-END ARCHITECTURE

**Commit → build → test → publish → deploy → verify**

**01 — Developer**
Writes code and commits changes.

**02 — GitHub repository**
Source code is pushed to the GitHub repository.

**03 — Jenkins Freestyle job on the same AWS EC2 instance**
A Freestyle job detects the change and starts the build.

**04 — Docker image build and HTTP smoke test**
Builds the Docker image and runs an HTTP smoke test.

**05 — Docker Hub image registry**
The image is tagged and pushed to Docker Hub.

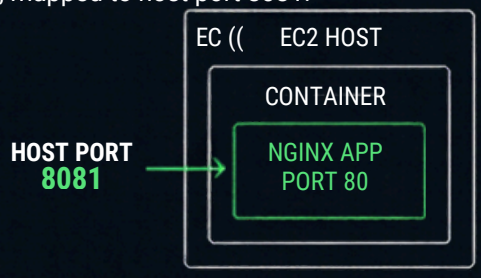
**06 — EC2 pulls and runs the image**
The EC2 instance pulls the latest image and runs the container.

**07 — Browser accesses the deployed website**
Users access the app through the public endpoint.

→ http://<EC2Public_IP>:8081

## APPLICATION CONTAINER (AS SHOWN IN SOURCE)

The source PDF shows an Nginx-based application container serving on internal port 80, mapped to host port 8081.




## TECHNICAL PORTS (AS SHOWN IN SOURCE)

| | |
|---|---|
| 22 | SSH administration |
| 8080 | Jenkins web interface |
| 8081 | deployed application host port |
| 80 | Nginx container port |
| 127.0.0.1:18081 | temporary smoke-test binding shown in the source |

## NEEDS CONFIRMATION

The source describes Nginx both as the application web server and as a reverse proxy. The supplied material clearly shows an Nginx application container on port 80, but it does not provide a separate reverse proxy configuration. Do not assume a second Nginx layer without verifying it.

## NAMING CHECK

The source contains similar but different repository strings. Confirm the canonical GitHub and Docker Hub names before configuring Jenkins.

# 03 //PREREQUISITES AND AWS SETUP

Prepare the host, restrict access, thon verify the runtime prerequisites

## PREREQUISITES CHECKLIST

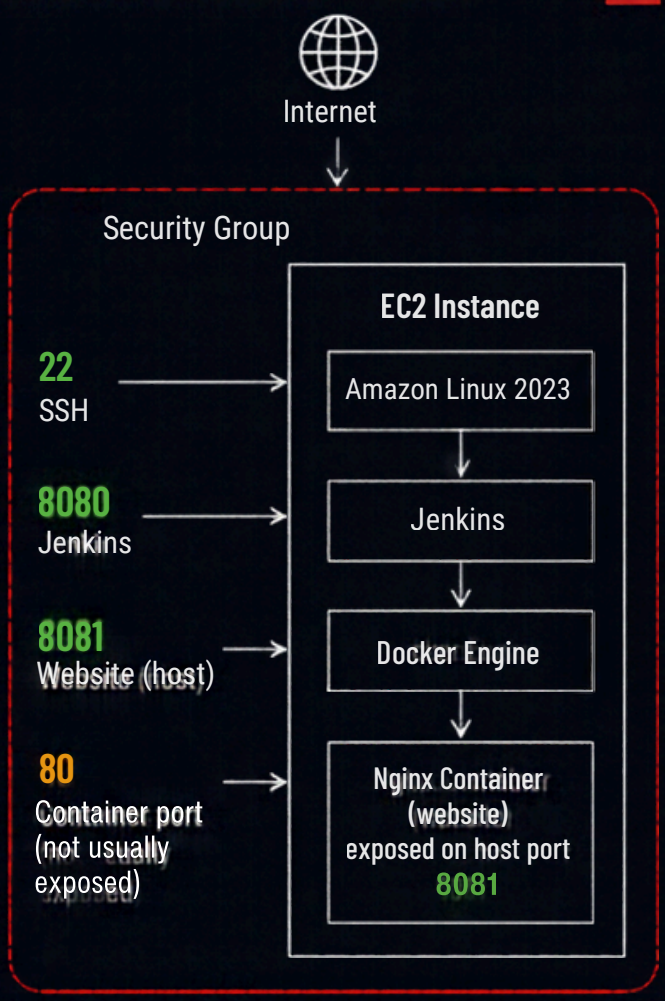
- [x] AWS account and EC2 instance
- [x] Amazon Linux 2023 host, as identified in the source
- [x] GitHub repository containing index.html and Dockerfile
- [x] Jenkins installed on the EC2 host
- [x] Docker Engine available to Jenkins
- [x] Docker Hub repository and access token
- [x] A confirmed public IP or DNS name

## INBOUND ACCESS PLAN (RECOMMENDED CONTROLS)

| | SERVICE | PURPOSE / ACCESS GUIDANCE |
|---|---|---|
| 22 | SSH | restrict to your administrator IP |
| 8080 | Jenkins | restrict to trusted administrator or VPN range |
| 8081 | deployed website | open only as required for the portfolio demo |
| 80 | container port | not normally exposed directly when using 8081:80 |

These are recommended access controls, not verified current rules.

## AWS EC2 SECURITY GROUP (EXAMPLE)




Do not expose SSH or Jenkins to 0.0.0/0.0 unless there is a documented, temporary reason. Prefer known source IPs, VPN access, or a controlled bastion path.

## SUBSTITUTE YOUR VALUES

- `<EC2_PUBLIC_IP>` — the public IP of your instance
- `<DOCKERHUB_USER>` — your Docker Hub namespace
- `<DOCKERHUB_TOKEN>` — a Docker Hub access token, never a password in a script
- `<GITHUB_REPOSITORY_URL>` — the confirmed repository URL

## SOURCE NOTE

The PDF includes security-group evidence, but this guide does not reproduce or validate the screenshots. Recheck the rules in AWS before access is enabled.

# 04 // JENKINS, REPOSITORY, AND CREDENTIALS

Configure the control plane without placing secrets in code

## 01 JENKINS READINESS

1. Provision or identify Jenkins on the EC2 host.
2. Confirm the Jenkins interface is intended to be reached on port 8080.
3. Complete Jenkins initial setup using the administrator workflow.

## 02 SOURCE REPOSITORY CONFIGURATION

4. Create a Freestyle project for the confirmed GitHub repository.
5. Set the repository URL and the intended branch or revision.
6. Add a build step that invokes the reviewed shell script on the EC2 host.

## 03 DOCKER HUB CREDENTIALS

- Store the Docker Hub username and access token in Jenkins Credentials.
- Bind them to the build only when needed.
- Use the token for authentication; do not place it in a shell script, screenshot, repository file, or command history.
- Use a least-privilege token and rotate it if exposed.

## REPOSITORY IDENTITY CHECK

The source text references 'ritesh-thorat-netfix-cicd' : another source panel shows 'ritesh-thorat-netflix-cicd'.

Treat the canonical name as unsolved until verified in GitHub.

Action: verify in GitHub

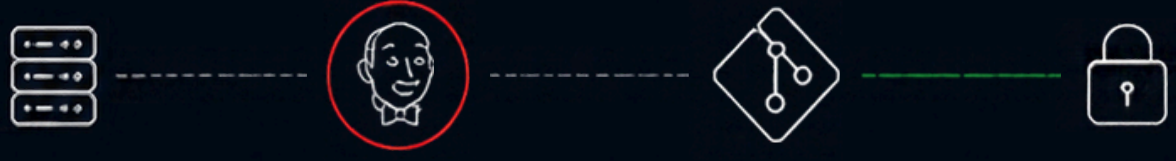
## SOURCE-GAP NOTE

The supplied PDF shows Jenkins readiness and a Freestyle-job concept, but it does not provide a complete, verified installation transcript or credential IDs.

**This guide intentionally does not invent them.**

## INPUTS TO CONFIRM

- <GITHUBREPOSITORY_URL>
- branch or revision
- <DOCKERHUBUSER>
- Jenkins credential IDs
- deployment container name



# 05 //FREESTYLE JOB AND BUILD/DEPLOY SCRIPT

05 / EXECUTE

USE PLACEHOLDERS, KEEP SECRETS OUT OF THE SCRIPT, AND VERIFY THE PORT MODEL BEFORE RUNNING

## FREESTYLE JOB SEQUENCE

1. Git checkout
2. Verify index.html and Dockerfile
3. Build image
4. Run temporary HTTP smoke test
5. Authenticate through Jenkins credential binding
6. Push image
7. Pull and run deployment container
8. Validate the public endpoint

## DOCKERFILE (SOURCE)

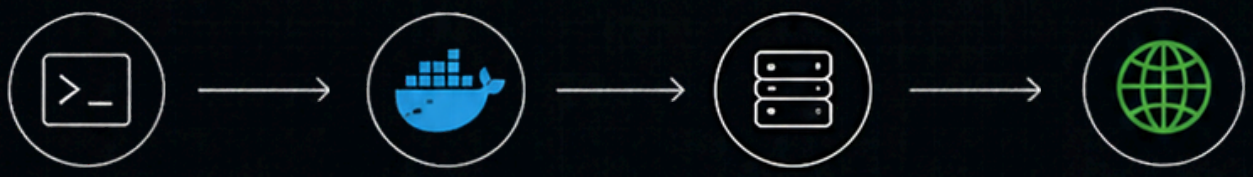
```
FROM nginx:alpine

COPY index.html /usr/share nginx.html/index.html

EXPOSE 80
```

This minimal Dockerfile serves the static site. The container listens on port 80.

## DOCKER IMAGE LIFECYCLE



**BUILD**
Local Build

**REGISTRY**
Docker Hub

**DEPLOY HOST**
Pull & Run

**PUBLIC ENDPOINT**
Validate

## REVIEWED COMMAND PATTERN – VERIFY NAMES AND PORTS BEFORE EXECUTION

```
IMAGE=DOCKERHUB_USER><REPOSITORY:latest
CONTAINER_NAME=netpix-app

docker build --pull -t "$IMAGE" .

docker run -d --name netpix-smoke -p 127.0.0.1:18081:80
"$IMAGE" curl -fsS http://127.0.0.1:18081/
docker rm -f netpix-smoke

docker push "$IMAGE"
docker pull "$IMAGE"
docker rm -f "$CONTAINERName" 2s/devnull || true
docker run -d --name "$CONTAINERName" -p 8081:80 "$IMAGE"
curl -fsS http://127.0.0.1:8081/
```

Replace <DOCKERHUB_USER> and <REPOSITORY> with confirmed values.

The source alternates between 18081 and 8081 for testing. This guide uses 18081 for a loopback smoke test and 8081 for the deployed host port as a proposed separation, confirm the intended project behavior before execution.

The source shows 'nginx:alpine' serving the application on container port 80.

Never add <DOCKERHUB_TOKEN> to this script.

**CAUTION**

**The exact existing container cleanup behavior, repository namespace, branch, and credential binding are not fully specified in the source. Review the script in a safe environment before enabling automatic deployment.**

# 06 // VERIFICATION AND TROUBLESHOOTING

## EVIDENCE TO COLLECT AFTER EACH STAGE, WITHOUT OVERSTATING RESULTS

### VERIFICATION CHECKLIST

- [ ] Source files present: index.html and Dockerfile
- [ ] Docker image builds without errors
- [ ] Temporary container returns an HTTP response
- [ ] Docker Hub authentication succeeds through Jenkins credential binding
- [ ] Image is pushed to the confirmed repository tag
- [ ] Deployment container is running on the EC2 host
- [ ] The application responds on the intended host port 8081
- [ ] The public endpoint uses http://<EC2Public_IP>8081

### >_ DIAGNOSTIC COMMANDS

Run these commands on the EC2 host to investigate containers, services, and connectivity.

```
docker ps
# List running containers

docker logs netpix-app
# View logs for the application container

curl -I http://127.0.0.1:8881/
# Check HTTP response from the local host port

sudo systemctl status docker
# Verify Docker service status
```

### TROUBLESHOOTING MATRIX

| SYMPTOM | INVESTIGATE |
|---|---|
| Port already allocated | inspect 'docker ps' and stop or rename the conflicting container |
| HTTP smoke test fails | inspect container logs and confirm the image serves on port 80 |
| Push denied | verify the Docker Hub namespace, token scope, and Jenkins credential binding |
| Public URL unreachable | check the EC2 security group and confirm host port 8081 is mapped |
| Jenkins cannot run Docker | verify Docker service status and the Jenkins account's Docker permissions |
| Repository checkout fails | verify the canonical GitHub URL, branch, and access configuration |

### SECURITY NOTE

Docker-group access can provide root-equivalent control on the host. Grant it only when required and treat Jenkins as a privileged automation service.

### SOURCE LIMITATIONS

The supplied PDF does not provide verified build durations, performance statistics, test output, or reproduable success logs. This guide does not claim them.

**BEFORE PORTFOLIO PUBLICATION:** . replace placeholders, remove secrets, verify URLs, and label screenshots as evidence only when they are current and authentic.